

Practical 5

Alternate Data Streams (ADS)

1. Aim

To practically demonstrate the creation of Alternate Data Streams (ADS) on a Windows virtual machine for the purpose of hiding files, and to explore techniques for detecting and countering such data obfuscation methods.

2. Objectives

- Understand the architecture of the NTFS file system regarding data streams.
- Create a standard text file and embed a hidden Alternate Data Stream within it.
- Verify that the primary file's size and attributes remain unchanged, masking the hidden data.
- Access and execute data hidden inside an ADS.
- Utilize built-in Windows commands (like `dir /r`) to detect hidden streams.

3. Tools Used

- **Operating System:** Windows 10/11 Virtual Machine (VM)
- **File System:** NTFS (New Technology File System)
- **Command-Line Interface:** Windows Command Prompt (cmd.exe)

4. System Requirements

- A working installation of Windows on a virtual environment (VirtualBox / VMware).
- The volume hosting the lab must be formatted as NTFS (FAT32 and exFAT do not support ADS).
- Administrative privileges are not strictly required for basic ADS creation, but helpful for advanced forensic analysis.

5. Theory

Alternate Data Streams (ADS) is a feature specific to the New Technology File System (NTFS) created by Microsoft. It was originally introduced to provide compatibility with the Hierarchical File System (HFS) used by Apple Macintosh computers, which uses "resource forks" and "data forks" to store file information.

In Windows, every file has at least one primary data stream (denoted as `$DATA`). ADS allows an arbitrary number of additional data streams to be attached to a single file or directory. Because standard Windows tools like Windows Explorer and the basic `dir` command only report the size of the primary data stream, any data placed in an alternate stream is effectively invisible to the casual observer. Threat actors often use ADS to hide malicious payloads, rootkits, or exfiltrated data.

6. Lab Setup

The lab is conducted entirely within an isolated Windows Virtual Machine. A dedicated folder named `C:\Labs\ADS_Demo` is created to ensure the environment is contained.

7. Procedure & Attack Flow

Step 1: Creating the Primary (Host) File

First, we create a normal, benign text file that will serve as the host for our hidden data. We check its contents to ensure it behaves normally.

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt" and has standard window controls (minimize, maximize, close) on the right. The command prompt shows the following sequence of commands and output:

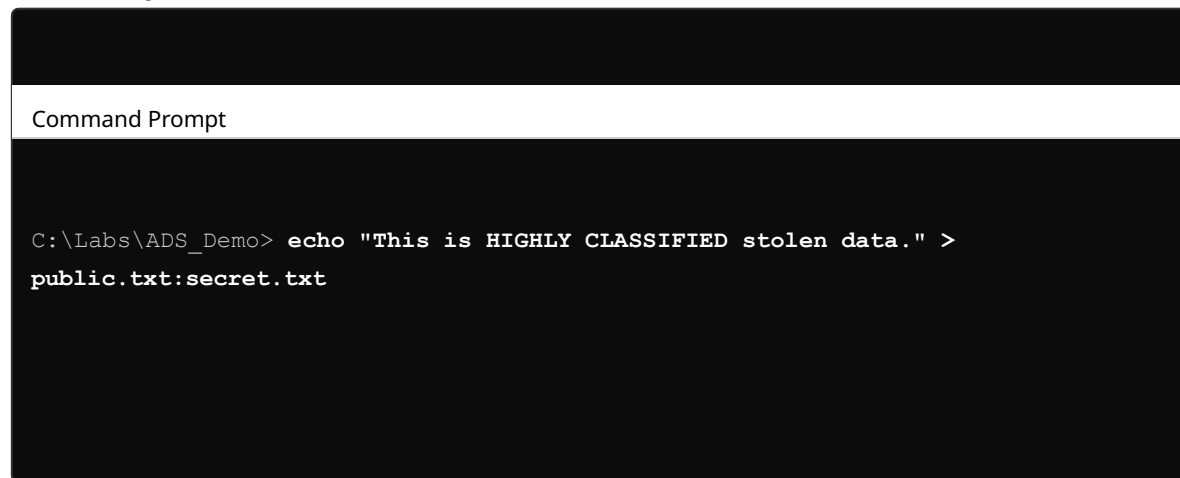
```
C:\Labs\ADS_Demo> echo "This is a normal, visible text file." > public.txt

C:\Labs\ADS_Demo> type public.txt
"This is a normal, visible text file."
```

Screenshot 1: Creating the initial standard text file

Step 2: Creating the Alternate Data Stream

Next, we attach a hidden data stream to `public.txt`. We do this by appending a colon (:) followed by the name of the stream (`secret.txt`) to the host file's name.



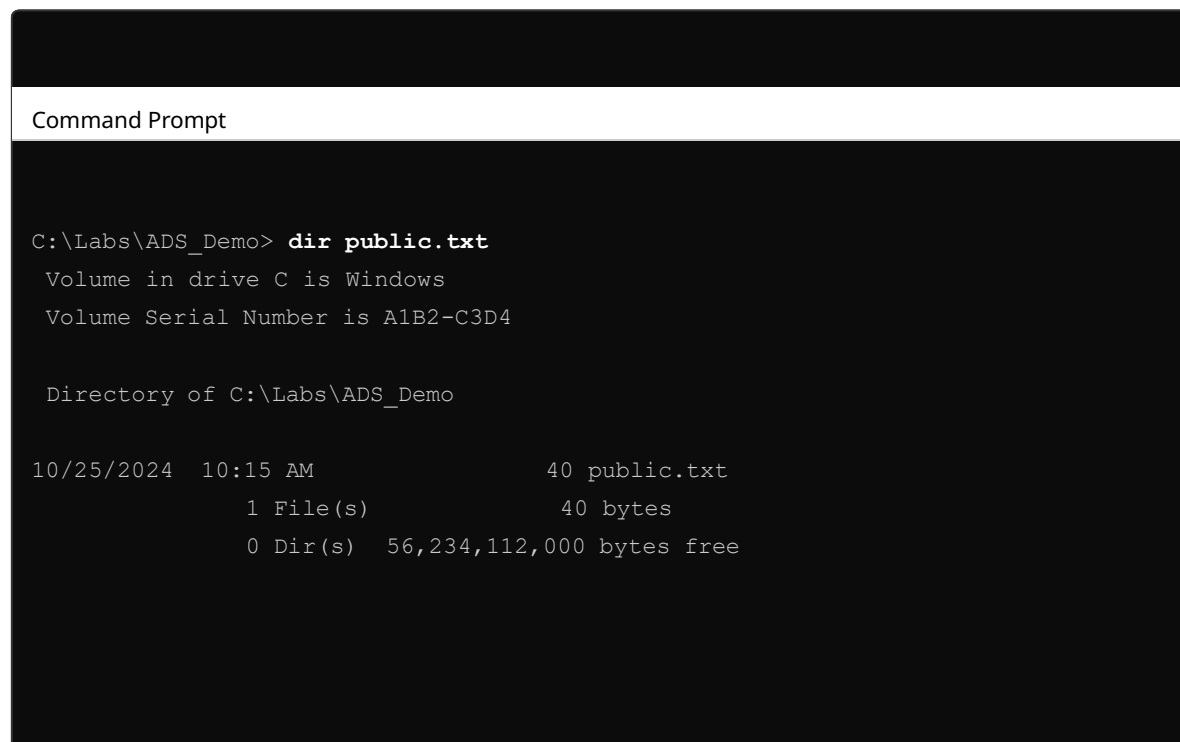
```
Command Prompt

C:\Labs\ADS_Demo> echo "This is HIGHLY CLASSIFIED stolen data." >
public.txt:secret.txt
```

Screenshot 2: Writing hidden text into the Alternate Data Stream

Step 3: Observing the Stealth Capabilities

We use the standard `dir` command to check the file size of `public.txt`. Even though we added a significant amount of text into the ADS, the reported file size of the host file does not change. It remains exactly as it was when initially created.



```
Command Prompt

C:\Labs\ADS_Demo> dir public.txt
Volume in drive C is Windows
Volume Serial Number is A1B2-C3D4

Directory of C:\Labs\ADS_Demo

10/25/2024  10:15 AM                40 public.txt
               1 File(s)                40 bytes
               0 Dir(s)  56,234,112,000 bytes free
```

Screenshot 3: The standard 'dir' command fails to show the hidden data size

Step 4: Accessing the Hidden Content

To read the hidden data, standard commands like `type` will not work directly on the stream. Instead, we use the `more` command or open it directly in applications like Notepad that support stream syntax.



```
Command Prompt

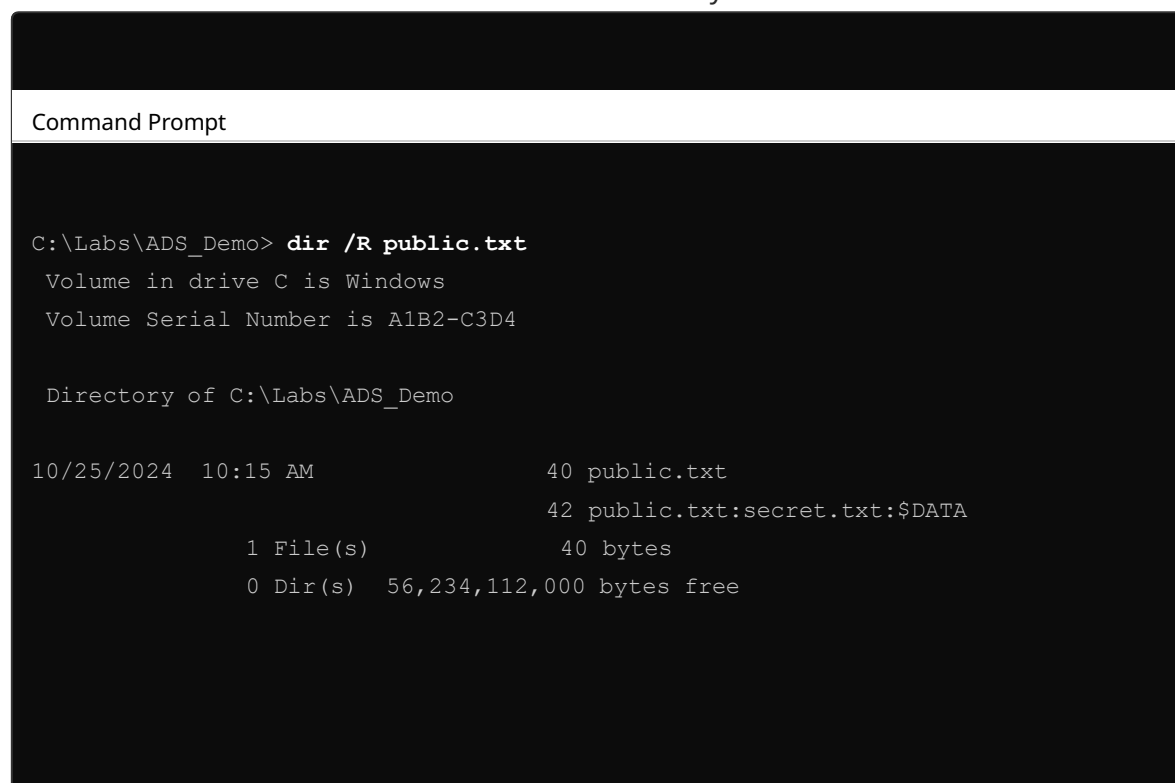
C:\Labs\ADS_Demo> more < public.txt:secret.txt
"This is HIGHLY CLASSIFIED stolen data."

C:\Labs\ADS_Demo> notepad public.txt:secret.txt
```

Screenshot 4: Accessing the hidden data using the 'more' command

Step 5: Detecting the Alternate Data Stream

To counter this hiding technique, system administrators and forensic analysts use the `/R` switch with the `dir` command. This switch forces the command prompt to display all alternate data streams attached to files in the directory.



```
Command Prompt

C:\Labs\ADS_Demo> dir /R public.txt
Volume in drive C is Windows
Volume Serial Number is A1B2-C3D4

Directory of C:\Labs\ADS_Demo

10/25/2024  10:15 AM                40 public.txt
                                42 public.txt:secret.txt:$DATA
               1 File(s)                40 bytes
               0 Dir(s)  56,234,112,000 bytes free
```

Screenshot 5: Successfully detecting the ADS using the 'dir /R' command

8. Observations

- The addition of the stream `secret.txt` to `public.txt` had absolutely no impact on the file size reported by standard GUI properties (Windows Explorer) or standard CLI commands.
- Opening `public.txt` normally only displays the benign text; the hidden text remains invisible to standard file opening procedures.
- The `dir /R` command clearly reveals the existence, name, and exact byte size of the hidden stream (labeled with `$DATA`).

9. Results

The practical execution of hiding data using Alternate Data Streams was successful. Data was successfully embedded behind a host file, completely bypassed standard surface-level size checks, and was subsequently retrieved and detected using forensic command-line switches.

10. Findings

- ADS is a powerful, built-in mechanism native to NTFS that requires no external third-party software to exploit.
- Because it is a native file system feature, traditional antivirus software historically struggled to scan streams, though modern EDR (Endpoint Detection and Response) solutions now inspect them routinely.
- Not only text, but fully executable payloads (e.g., `.exe` files) can be hidden inside an ADS.

11. Limitations

- ADS is strictly an NTFS feature. If a file containing an ADS is moved or copied to a non-NTFS volume (like a FAT32 USB drive), sent via email, or uploaded to cloud storage, the alternate streams are automatically stripped and the hidden data is permanently lost.
- Hidden executables in modern Windows versions cannot be executed directly via the command line syntax (e.g., `start file.txt:malware.exe`) due to modern security patches, requiring workarounds like WMI or PowerShell for execution.

12. Precautions

- When performing forensic imaging, always ensure the imaging tool captures logical file system features (like ADS) rather than just standard files.
- Security analysts should regularly audit critical directories using tools like `dir /R`, PowerShell's `Get-Item -Stream *`, or Sysinternals Streams utility.

13. Conclusion

This laboratory exercise effectively demonstrated the dual nature of Alternate Data Streams: as a legitimate file system feature and as a covert channel for malicious data hiding. By understanding how attackers leverage ADS to conceal payloads without alerting standard file size checks, security professionals can better implement proper detection mechanisms, utilizing switches like `/R` or dedicated forensic tools to uncover hidden artifacts during incident response.